

Lab 11 · El registro y el cuadro de luces

Curso de Spring Boot · Servicio de Impuestos Internos · 2026

75 minutos · Spring Boot 4.1.0 · Java 25 (Temurin) · PostgreSQL 16 embebido

Resumen

Que una aplicación tiene que poder contar qué le pasa. Con la base caída: **liveness 200**, **readiness 503**, y un health que **nombra la causa** en vez de decir sólo «DOWN».

Índice

1. Antes de empezar	2
1.1. Qué vas a lograr	2
1.2. Qué necesitas tener listo	2
1.3. Cómo copiar el código de esta guía	2
1.4. La puesta a punto	3
2. El caso	3
2.1. El registro y el cuadro de luces, que es la metáfora de este laboratorio	3
3. Los pasos	3
3.1. Paso 1 · Actuator, y qué NO se expone	3
3.2. Paso 2 · Seguir UNA petición entre miles	5
3.3. Paso 3 · Una métrica que le importa al negocio	7
3.4. Paso 4 · Un health que dice QUÉ se rompió	8
3.5. Paso 5 · Liveness no es readiness	10
4. Lo que aprendiste	12
5. Para profundizar	13
6. Antes de cerrar	13

1 Antes de empezar

1.1 Qué vas a lograr

Hasta ahora, cuando querías saber si algo funcionaba, **mirabas**. En producción no hay nadie mirando: hay un orquestador que decide solo si tu aplicación recibe tráfico o si la reinicia.

Hoy vas a hacer que la aplicación **hable**: que se pueda seguir una petición entre miles, que cuente algo que le importa al negocio, y que diga con precisión **qué** se rompió — y sobre todo, que distinga entre «estoy muerto, reiníciame» y «estoy vivo pero no puedo atender».

1.2 Qué necesitas tener listo

Requisito	Cómo lo compruebas	Qué tiene que salir
Los labs 01 y 04 hechos	Sabes crear endpoints y usar una base	—
Estar en la carpeta del lab	cd labs/lab-11-observabilidad/practica	El cd no da error

1.3 Cómo copiar el código de esta guía

Al copiar de un PDF se pierden los espacios del principio de línea, y a veces una línea larga se parte en dos. Con Java no importa; **con el `application.yml` sí**, y por eso aquí nunca se te va a pedir pegar un bloque de YAML entero sin decírtelo. El código completo está en `labs/lab-11-observabilidad/solucion/`.

Este lab pega tres trozos distintos bajo la misma clave `management`: del `application.yml`, en los pasos 1, 4 y 5. **Es una sola clave, no tres:** el segundo trozo y el tercero se **funden** con lo que ya escribiste. Un `application.yml` con `management`: repetido tres veces no es válido y la aplicación no arranca.

1.4 La puesta a punto

```
cd labs/lab-11-observabilidad/practica
./mvnw spring-boot:run
```

Escucha en el **8101** y su PostgreSQL en el **55442**. **Párala con Ctrl+C.**

2 El caso

La oficina de la DGT funciona. Nadie sabe **cómo** funciona: cuántos trámites emite, si la conexión con el archivo del sótano está viva, ni qué le pasó exactamente a la petición del ciudadano que llamó para quejarse.

2.1 El registro y el cuadro de luces, que es la metáfora de este laboratorio

Toda oficina tiene un libro de registro y un cuadro de luces en la entrada.

El libro de registro anota lo que va pasando. Sirve de poco si las anotaciones no se pueden seguir: con doscientas personas atendidas a la vez, «se emitió un trámite» no dice **de quién**. Por eso cada expediente lleva **un número que se apunta en todas sus líneas** — y así se puede seguir uno solo entre miles. Eso es el *trace id*.

El contador de la puerta cuenta trámites emitidos. No es un dato técnico: es un dato que le importa al director de la oficina.

Y el cuadro de luces dice si la oficina puede atender. Tiene **dos luces distintas**, y confundirlas es el error del día:

- **¿Está el edificio en pie?** Si no, hay que **reconstruirlo** — reiniciar el proceso.
- **¿Puede atender público ahora mismo?** El edificio está en pie, pero el archivo del sótano está inundado: **cierra la puerta al público** y sigue en pie, esperando a que el sótano se seque.

Reiniciar una oficina cuyo problema es que el sótano está inundado **no arregla nada**, y encima tira lo que estaba a medias. Por eso las dos luces son distintas.

3 Los pasos

3.1 Paso 1 · Actuator, y qué NO se expone

Qué vamos a hacer

Encender los endpoints de diagnóstico, y **elegir a mano** cuáles.

Para entenderlo mejor

Poner el cuadro de luces en la entrada. Con cuidado: hay indicadores que no deben verse desde la calle.

El problema

Sin nada que consultar, saber si la aplicación está bien exige entrar a la máquina. Y el orquestador que decide si la reinicia no puede entrar a mirar: necesita **una URL**.

La alternativa, y por qué no

- **Escribir tu propio /estado**: lo hace todo el mundo al principio, y acaba siendo un endpoint que sólo dice «ok» porque respondió.
- **Actuator con include**: `"*"`: enciende **todo**, y ahí dentro hay endpoints que listan las variables de entorno, los beans, la configuración con sus valores, y que permiten cambiar niveles de log en caliente. En una aplicación expuesta, eso es un mapa del edificio.
- **Actuator con lista blanca nominal**, que es lo de aquí: se nombran uno a uno los que se quieren. Añadir uno nuevo pasa a ser **una decisión**, no un descuido.

Se pega

En `practica/src/main/resources/application.yml`, donde dice `#` escribe aquí:

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics
```

Se corre

```
curl -s localhost:8101/actuator
```

Lo que vas a ver

```
['self', 'info', 'health', 'health-path', 'metrics', 'metrics-requiredMetricName']
```

Sólo lo que nombraste. Prueba a pedir `/actuator/env` o `/actuator/beans`: 404. Están apagados, y eso es lo correcto.

Vas bien si...

`/actuator` lista sólo `health`, `info` y `metrics`. Si vieras veinte entradas, tienes `"*"` puesto.

Si te atascas

1 · EL PUERTO 55442 YA ESTA OCUPADO O ESTE MISMO PROYECTO YA ESTA CORRIENDO

Los dos candados del Lab 04:

```
lsof -ti:55442 | xargs kill -9
```

2 · /actuator devuelve 404.

Falta la dependencia de Actuator en el `pom.xml`, o el bloque `management`: quedó mal indentado.

3 · La aplicación no arranca y habla del YAML.

Se perdió la sangría al pegar. Cada nivel son **dos espacios**, nunca tabulador.

3.2 Paso 2 · Seguir UNA petición entre miles

Qué vamos a hacer

Poner un identificador por petición que aparezca en **todas** sus líneas de log.

Para entenderlo mejor

El número de expediente. Se apunta en cada línea que se escriba sobre ese trámite, y así se puede seguir uno concreto en un libro con miles de anotaciones.

El problema

Con varias peticiones a la vez, las líneas de log se **entrelazan**. «Emitiendo trámite» y «Trámite emitido» pueden ser de peticiones distintas, y no hay forma de saberlo.

La alternativa, y por qué no

- **Pasar el id como argumento** a cada método y a cada `log.info`: hay que tocar **todas** las firmas del proyecto por un dato que no es del dominio, y basta olvidar un `log.info` para perder el hilo justo donde hacía falta.
- **El MDC**, que es lo de aquí: se pone una vez en un filtro, queda atado al hilo, y a partir de ahí **toda** línea lo lleva sin que nadie lo mencione.
- **OpenTelemetry**: hace esto y además propaga entre servicios y da tiempos por tramo. Es lo correcto en un sistema real y es otra pieza que instalar; el MDC no necesita nada.

Y el detalle que hay que decir en voz alta: el filtro **quita** el id al terminar, en un `finally`. Los hilos se reutilizan, y sin eso la petición siguiente **hereda** el id de la anterior — una fuga que se ve en producción como logs que mienten.

Se pega

Archivo **nuevo** `practica/src/main/java/cl/dgt/observabilidad/infra/FiltroDeCorrelacion.java` — el archivo entero:

```
package cl.dgt.observabilidad.infra;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.slf4j.MDC;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.UUID;

@Component
public class FiltroDeCorrelacion extends OncePerRequestFilter {
```

```

public static final String CABECERA = "X-Trace-Id";
public static final String CLAVE = "traceId";

@Override
protected void doFilterInternal(HttpServletRequest petition, HttpServletResponse
    ↪ respuesta,
                                FilterChain cadena) throws ServletException,
    ↪ IOException {
    String id = petition.getHeader(CABECERA);
    if (id == null || id.isBlank()) {
        id = UUID.randomUUID().toString().substring(0, 8);
    }
    MDC.put(CLAVE, id);
    respuesta.setHeader(CABECERA, id);
    try {
        cadena.doFilter(petition, respuesta);
    } finally {
        MDC.remove(CLAVE);
    }
}
}

```

Y en application.yml, el patrón que lo imprime — **fúndelo con el bloque logging: que ya está:**

```

logging:
  pattern:
    console: "%d{HH:mm:ss.SSS} %-5level [%X{traceId:-.....}] %logger{25} - %msg%n"

```

Se corre

```

curl -i -X POST localhost:8101/tramites \
  -H 'Content-Type: application/json' \
  -d '{"tipo":"F29","rut":"76.543.210-K"}'

```

Lo que vas a ver

En la respuesta, la cabecera:

```

HTTP/1.1 200
X-Trace-Id: 40095b6d

```

Y en la consola del servidor, **las dos líneas con el mismo número:**

```

01:13:30.237 INFO [40095b6d] c.d.o.c.TramiteController - Emitiendo trámite tipo=F29
    ↪ rut=76.543.210-K
01:13:30.265 INFO [40095b6d] c.d.o.c.TramiteController - Trámite 1 emitido

```

El ciudadano se puede llevar ese número. Cuando llame a reclamar, con 40095b6d se encuentra su petición entera en el registro.

Tu id va a ser otro, y cambia en cada petición. Lo que importa es que **las dos líneas lleven el mismo**, y que la cabecera de la respuesta lo devuelva.

Vas bien si...

Las dos líneas de log de una misma petición llevan el mismo id entre corchetes, y la respuesta trae X-Trace-Id.

Si te atascas

1 · Los corchetes salen con puntos: [.....]

El filtro no se está ejecutando. Comprueba que tiene `@Component` y que está bajo `cl.dgt.observabilidad`.

2 · Sale el id, pero la petición siguiente lleva el mismo.

Falta el `MDC.remove(...)` en el `finally`. Es la fuga que hace que los logs mientan.

3 · No aparece el corchete en ninguna línea.

Falta el patrón en `application.yml`, o lo pegaste fuera del bloque `logging`.

3.3 Paso 3 · Una métrica que le importa al negocio

Qué vamos a hacer

Contar los trámites emitidos, y exponerlo.

Para entenderlo mejor

El contador de la puerta. No cuenta peticiones HTTP ni uso de memoria: cuenta **trámites**, que es lo que le importa a quien dirige la oficina.

El problema

Actuator ya trae métricas técnicas —memoria, hilos, peticiones por segundo— y ninguna responde a «¿cuántos trámites llevamos hoy?». Esa la tiene que declarar quien conoce el negocio.

La alternativa, y por qué no

- **Un Gauge**: refleja un valor que sube y baja. Los trámites emitidos **no bajan nunca**, así que sería mentir sobre la naturaleza del dato — y las herramientas de métricas tratan los dos tipos de forma distinta al agregar.
- **Un Timer**: mide además cuánto tarda, que muchas veces es más útil.
- **Un Counter**, que es lo de aquí, **declarado una vez en el constructor**. Buscarlo en el registro en cada petición funciona; con veinte métricas, se nota.

Se pega

En `practica/src/main/java/cl/dgt/observabilidad/controllers/TramiteController.java`, arriba con los `import`:

```
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
```

El campo **debajo** de `private final TramiteRepository repositorio;`, y el constructor **reemplazando el que hay** — le entra un parámetro más:

```
private final Counter emitidos;

public TramiteController(TramiteRepository repositorio, MeterRegistry registro) {
    this.repositorio = repositorio;
    this.emitidos = Counter.builder("dgt.tramites.emitidos")
        .description("Trámites emitidos desde que arrancó la aplicación")
        .register(registro);
}
```

Y donde el controlador emite, la línea que incrementa:

```
emitidos.increment();
```

Lo que vas a ver

```
curl -s localhost:8101/actuator/metrics/dgt.tramites.emitidos
```

```
{
  "name": "dgt.tramites.emitidos",
  "description": "Trámites emitidos desde que arrancó la aplicación",
  "measurements": [ { "statistic": "COUNT", "value": 1.0 } ]
}
```

Vas bien si...

La métrica existe y su valor sube cada vez que emites un trámite.

Si te atascas

1 · 404 al pedir la métrica.

La métrica no existe hasta que se **registra**, y se registra al construir el controlador. Si nunca has emitido un trámite ni arrancado con el contador declarado, no está.

2 · El valor no sube.

Falta el `emitidos.increment()` en el método que emite.

3.4 Paso 4 · Un health que dice QUÉ se rompió

Qué vamos a hacer

Escribir un indicador de salud propio que consulte la base **de verdad** y nombre el problema.

Para entenderlo mejor

Una luz que sólo dice «avería» obliga a bajar al sótano a mirar. Y quien baja a mirar suele hacerlo a las tres de la mañana.

El problema

El health que trae Spring de fábrica comprueba que la conexión responda, y contesta UP o DOWN. Cuando dice DOWN, no dice **por qué** ni **cuánto tardó**.

La alternativa, y por qué no

- **El DataSourceHealthIndicator de fábrica:** gratis y suficiente para saber si el pool responde. No ejecuta nada del dominio.
- **Un indicador propio**, que es lo de aquí: ejecuta una consulta real contra una tabla que te importa y **nombra la causa**.

Cuidado con lo que se consulta: esto se ejecuta cada pocos segundos. Un `count(*)` sobre una tabla enorme convierte el health en una consulta cara.

Y el nombre del bean se pone **a mano** —`@Component("baseDeDatos")`— porque el grupo del paso 5 lo referencia por ese nombre: si alguien renombra la clase al refactorizar, el grupo se queda apuntando a nada y **no falla** — simplemente deja de vigilar la base.

Se pega

En `application.yml`, **fundido con el management: del paso 1:**

```
endpoint:  
  health:  
    show-details: always
```

Y el archivo **nuevo** `practica/src/main/java/cl/dgt/observabilidad/infra/SaludDeLaBase.java` — entero:

```
package cl.dgt.observabilidad.infra;  
  
import org.springframework.boot.health.contributor.Health;  
import org.springframework.boot.health.contributor.HealthIndicator;  
import org.springframework.stereotype.Component;  
  
import javax.sql.DataSource;  
import java.sql.Connection;  
import java.sql.SQLException;  
import java.sql.Statement;  
  
@Component("baseDeDatos")  
public class SaludDeLaBase implements HealthIndicator {  
  
    private final DataSource dataSource;  
  
    public SaludDeLaBase(DataSource dataSource) {  
        this.dataSource = dataSource;  
    }  
  
    @Override  
    public Health health() {  
        long inicio = System.currentTimeMillis();  
        try (Connection conexion = dataSource.getConnection();
```

```

        Statement sentencia = conexion.createStatement() {

        sentencia.execute("SELECT count(*) FROM tramite");

        return Health.up()
            .withDetail("consulta", "SELECT count(*) FROM tramite")
            .withDetail("milisegundos", System.currentTimeMillis() - inicio)
            .build();

    } catch (SQLException e) {
        return Health.down()
            .withDetail("causa", "la base de datos no responde")
            .withDetail("detalle",
                ↪ e.getMessage().lines().findFirst().orElse(""))
            .withDetail("milisegundos", System.currentTimeMillis() - inicio)
            .build();
    }
}
}

```

Lo que vas a ver

```
curl -s localhost:8101/actuator/health
```

```

"baseDeDatos": {
  "status": "UP",
  "details": { "consulta": "SELECT count(*) FROM tramite", "milisegundos": 1 }
}

```

Dice qué comprobó y cuánto tardó. Y cuando falle, dirá por qué — lo vas a ver en el paso 5.

Vas bien si...

/actuator/health incluye un componente baseDeDatos con details. Si no ves los detalles, falta show-details: always.

Si te atascas

1 · Sale "status": "UP" pero sin details.

Falta show-details: always, o lo pegaste como un management: nuevo en vez de fundirlo con el del paso 1 — y entonces el archivo no es válido o una clave pisa a la otra.

2 · No aparece el componente baseDeDatos.

El nombre del bean no es ése. Va en la anotación: @Component("baseDeDatos").

3.5 Paso 5 · Liveness no es readiness

Qué vamos a hacer

Encender las dos sondas, meter la base **en la de readiness y no en la de liveness**, y tirar la base para ver la diferencia.

Para entenderlo mejor

Las dos luces del cuadro:

- **Liveness** — «¿está el edificio en pie?» Si contesta que no, la respuesta correcta es **reiniciar el proceso**.
- **Readiness** — «¿puede atender público?» Si contesta que no, la respuesta correcta es **sacarlo de la rotación** y dejarlo tranquilo hasta que se recupere.

El problema

Si metes la base en liveness, pasa esto: se cae la base, el orquestador cree que tu aplicación está muerta y **la reinicia**. La base sigue caída, así que la reinicia otra vez. Y otra. **Un bucle de reinicios que no arregla nada y tira lo que hubiera a medias**.

La alternativa, y por qué no

- **Una sola sonda para todo**: es lo que hay por defecto si no separas, y lleva justo al bucle de arriba.
- **Las dos separadas**, que es lo de aquí, con el criterio: en **liveness** va sólo lo que se arregla reiniciando; en **readiness**, todo aquello sin lo cual no puedes atender.

La base de datos **no se arregla reiniciando tu proceso**. Va en readiness.

Se pega

En application.yml, **fundido otra vez con el mismo management::**

```
probes:
  enabled: true
group:
  readiness:
    include: readinessState,baseDeDatos
  liveness:
    include: livenessState
```

Se corre

Con la base viva:

```
curl -o /dev/null -s -w 'liveness %{http_code}\n'
↪ localhost:8101/actuator/health/liveness
curl -o /dev/null -s -w 'readiness %{http_code}\n'
↪ localhost:8101/actuator/health/readiness
```

Y ahora **se tira la base**:

```
curl -X POST localhost:8101/simulador/base-caida
```

Lo que vas a ver

Con la base viva:

```
liveness 200
readiness 200
```

Con la base caída:

```
liveness 200    <- el proceso está bien. NO lo reinicies
readiness 503   <- no puede atender. Sácalo de rotación
```

Y el health, nombrando la causa:

```
"baseDeDatos": {
  "status": "DOWN",
  "details": {
    "causa": "la base de datos no responde",
    "detalle": "HikariPool-1 - Connection is not available, request timed out after
      ↪ 2010ms",
    "milisegundos": 2010
  }
}
```

Aquí está el laboratorio entero en cinco líneas. El proceso está sano, no puede atender, y dice exactamente por qué y cuánto esperó.

Vuelve a levantarla:

```
curl -X POST localhost:8101/simulador/base-sana
```

Vas bien si...

Con la base caída obtienes **200 en liveness y 503 en readiness**, y el health nombra la causa.

Si te atascas

1 · Los dos dan 503.

Metiste baseDeDatos también en el grupo de liveness. Ése es exactamente el error que lleva al bucle de reinicios.

2 · /actuador/health/liveness da 404.

Falta probes: enabled: true.

3 · Readiness sigue en 200 con la base caída.

El grupo no incluye baseDeDatos, o el nombre no coincide con el del @Component.

4 Lo que aprendiste

1 · Sin un identificador por petición, el registro no sirve con tráfico.

El MDC lo pone una vez y lo lleva toda la petición. Y hay que quitarlo al terminar, o el siguiente hereda el de antes.

2 · Las métricas que importan las declara quien conoce el negocio.

Actuator cuenta memoria e hilos. «Cuántos trámites llevamos» lo tiene que contar alguien, y es un Counter porque los trámites emitidos no bajan.

3 · Un health que sólo dice DOWN obliga a alguien a ir a mirar.

Uno que nombra la causa y el tiempo se arregla desde la pantalla. Y cuidado con lo que consulta: se ejecuta cada pocos segundos.

4 · Liveness y readiness responden preguntas distintas.

Liveness: ¿reinicio? Readiness: ¿le mando tráfico? Meter la base en liveness produce un bucle de reinicios que no arregla la base y sí tira lo que estaba a medias.

5 Para profundizar

- **Pon include:** "*" en la exposición y mira /actuator/env. Después quítalo y piensa qué habría pasado si eso estuviera abierto en internet.
- **Manda tu propia cabecera X-Trace-Id** en un curl y comprueba que el filtro la respeta en vez de inventar una.
- **Quita el MDC.remove** y lanza dos peticiones seguidas. Mira los ids.
- **Mete baseDeDatos en el grupo de liveness** y tira la base. Imagina eso en producción con un orquestador delante.
- **Cambia la consulta del health** a algo caro y mira los milisegundos.

6 Antes de cerrar

Deja la base sana y para la aplicación con Ctrl+C:

```
curl -X POST localhost:8101/simulador/base-sana
./mvnw clean
```

Lo que te llevas:

Una petición se sigue por su id; una métrica de negocio la declara quien conoce el negocio; un health dice qué se rompió; y liveness y readiness no son la misma pregunta.

Lo que queda pendiente, y abre el Lab 12: todo lo que hace tu aplicación lo hace **porque alguien se lo pidió**. Hay trabajo que tiene que ocurrir solo —de madrugada, cada diez minutos— y trabajo que no debería hacer esperar a quien está en la ventanilla. En el Lab 12 la aplicación empieza a trabajar sola.